P4 - 2023 Exam: Library system with a simulated "ChatGPT" librarian

*25 marks: DB + validation (7) + the librarian function (7) + GUI (11). The GUI is gone; the **validation rules and the question-parsing function** are the transferable parts. Files:* `Solutions_py/2023_library_librarian_solution.py` *(window) and* `2023_library_console.py` *(terminal, practise this).*

1. What the paper asked

Task	Requirement	Marks
1a	Screen: ISBN, Title, Author, Year, Save, Clear, Display Books, Ask Librarian	5
1a	Save valid book into database **LibraryDB**; show a success message	4
1a	**Clear** empties ALL windows	2
1a	**Display Books** shows every saved book	3
1b	No duplicate **ISBN** (message)	1
1b	Year not in the future AND not earlier than **1450**	1
1b	Title and author not empty	1
1c	"Recommend me a book by [author]." answered from the DB	3.5
1c	"When was [book title] published." answered from the DB	3.5

Only those two sentence shapes need supporting.

2. Analogy

The library has a **catalogue card drawer** (the table). The ISBN is the card's unique number, so the drawer itself refuses a second card with the same ISBN (`PRIMARY KEY`). The "librarian" is a small function that **reads your sentence, spots which of the two patterns it is, pulls out the interesting part (the author or the title), looks it up in the drawer, and writes a reply.** There is no real AI: it is pattern matching plus a query, exactly what the paper means by "simulated ChatGPT, do not use an API".

3. Validation, line by line (terminal version)

```
import datetime
import re
import sqlite3

conn = sqlite3.connect("LibraryDB.db")
cursor = conn.cursor()
cursor.execute("""CREATE TABLE IF NOT EXISTS Books (
                      isbn   TEXT PRIMARY KEY,
                      title  TEXT NOT NULL,
                      author TEXT NOT NULL,
                      year   INTEGER NOT NULL)""")
conn.commit()
```

```
def add_book():
    isbn = input("ISBN: ").strip()
    title = input("Title: ").strip()
    author = input("Author: ").strip()
    year_text = input("Year: ").strip()

    if isbn == "" or title == "" or author == "":
        print("Error: ISBN, title and author cannot be empty!")
        return
    if not year_text.isdigit():                 # also rejects '-5' and 'abc'
```

```
        print("Error: invalid year of publication!")
        return
    year = int(year_text)
    if year < 1450 or year > datetime.date.today().year:
        print("Error: invalid year of publication!")
        return
    cursor.execute("SELECT 1 FROM Books WHERE isbn = ?", (isbn,))
    if cursor.fetchone() is not None:
        print("Error: this ISBN already exists!")
        return

    cursor.execute("INSERT INTO Books VALUES (?, ?, ?, ?)",
                   (isbn, title, author, year))
    conn.commit()
    print("Book added successfully!")
```

- `isbn TEXT PRIMARY KEY`: ISBNs contain dashes (`1-86092-038-1`), so store them as **text**, not numbers. PRIMARY KEY = unique + indexed.
- `year_text.isdigit()`: True only if every character is 0-9 (rejects `abc`, `-5`, `20.5`, blank). Then `int(...)`.
- `datetime.date.today().year`: today's year, so the "not in the future" rule never goes out of date (do **not** hard-code 2023).
- `year < 1450 or year > current`: invalid. Equivalent keep-form: `1450 <= year <= this_year` (Python lets you chain comparisons).
- The duplicate check is a `SELECT 1 ... WHERE isbn = ?`; `fetchone()` not `None` = already there.
- `return` **after each error message** = stop immediately, nothing is saved. Without `return`, the invalid book would still be inserted.

4. The virtual librarian, line by line

```
def virtual_librarian(question):
    """Two sentence shapes only: recommend by author / publication year."""
    match = re.match(r"recommend me a book by (.+?)[.?!]*$", question.strip(),
                     re.IGNORECASE)
    if match:
        author = match.group(1).strip()
        cursor.execute("SELECT title, author FROM Books "
                       "WHERE LOWER(author) = LOWER(?) "
                       "ORDER BY RANDOM() LIMIT 1", (author,))
        row = cursor.fetchone()
        return (f"How about '{row[0]}' by {row[1]}?" if row
                else f"Sorry, I have no books by {author}.")

    match = re.match(r"when was (?:published )?(.+?)(?: published)?[.?!]*$",
                     question.strip(), re.IGNORECASE)
    if match:
        title = match.group(1).strip()
        cursor.execute("SELECT title, year FROM Books "
                       "WHERE LOWER(title) = LOWER(?)", (title,))
        row = cursor.fetchone()
        return (f"{row[0]} was published in {row[1]}." if row
                else f"Sorry, I could not find '{title}'.")

    return "Sorry, I can only answer two kinds of question."
```

Regular expressions in plain English. `re.match(pattern, text)` asks "does the text start with this pattern?" Special pieces:

Piece	Means
`recommend me a book by`	those exact words (case ignored because of `re.IGNORECASE`)
`(.+?)`	**capture group**: "one or more of any characters", as few as needed. This is the part we want (the author)

Piece	Means
`[.?!]*$`	optional trailing `.`, `?` or `!`, then the end of the text
`match.group(1)`	the text caught by the first `( )`
`(?:published )?`	"the word *published* may appear here, but don't capture it"

- `LOWER(author) = LOWER(?)`: user typing `arthur conan doyle` still matches `Arthur Conan Doyle`.
- `ORDER BY RANDOM() LIMIT 1`: if the author has several books, pick one at random: that is the "recommendation".
- The conditional expression `(A if row else B)`: "give A if a row was found, otherwise B". The reply still gives a polite answer when nothing is found. **Always handle "not found"**: it is the classic lost mark.
- The sample screenshots use `when was published The Signalman?` (word order swapped from the question text) so the pattern accepts **both** orders.

If you do not trust regular expressions in the exam, a simpler (also valid) way:

```
q = question.lower().strip().rstrip("?.!")
if q.startswith("recommend me a book by "):
    author = q[len("recommend me a book by "):]      # slice off the prefix
elif q.startswith("when was ") and q.endswith(" published"):
    title = q[len("when was "):-len(" published")]     # slice both ends
```

5. Traps

- **TRAP:** `INSERT` of a duplicate ISBN on a PRIMARY KEY table raises `sqlite3.IntegrityError`. Your own `SELECT` check first gives a friendly message instead of a crash.
- **TRAP:** matching titles case-sensitively: `When was the signalman published` finds nothing. Use `LOWER()`.
- **TRAP:** using `=` for the author but the user typed extra spaces: `.strip()` the captured text.
- **TRAP:** f-string quotes: `f"How about '{title}' by {author}?"`: single quotes inside double quotes is fine.
- **TRAP:** year stored as text so `year > 2000` compares strings. Declare `year INTEGER` and convert with `int()`.

6. Anticipated questions

1. *Add a third question: "How many books are in the library?"* `SELECT COUNT(*) FROM Books`.
2. *Add: "List all books published before 1900."* `SELECT title FROM Books WHERE year < 1900 ORDER BY year`.
3. *Add: "Who wrote [title]?"* `SELECT author FROM Books WHERE LOWER(title)=LOWER(?)`.
4. *Delete a book by ISBN.* `DELETE FROM Books WHERE isbn = ?` then `commit()`; report if `cursor.rowcount == 0`.
5. *Why PRIMARY KEY on ISBN?* Guarantees uniqueness and fast lookup.
6. *Is this really ChatGPT?* No: pattern matching + SQL. Real systems call an LLM API, which the paper forbids.
7. *What if two books have the same title?* `fetchone()` returns only the first; use `fetchall()` and list every year.

7. Quick-reference box

```
year ok    1450 <= int(y) <= datetime.date.today().year
dup ISBN   SELECT 1 FROM Books WHERE isbn = ?   -> fetchone() is not None
parse      m = re.match(r"recommend me a book by (.+?)[.?!]*$", q, re.IGNORECASE)
           m.group(1)    # the captured author
lookup     WHERE LOWER(author) = LOWER(?)   ORDER BY RANDOM() LIMIT 1
not found  ALWAYS return a polite message
```